



Slides for P3724R1 — Integer division

Document number:

P3895R0

Date:

2025-03-11

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/intdiv-slides.cow



IMPORTANT: This document has custom controls:

-   : go to the next slide
-   : go to previous slide

Integer division

P3724R1

Introduction

- C++ supports integer division with rounding towards zero
- many other rounding modes are useful:

```
int bucket_size = 1000, elements = 100;  
  
int buckets_req = elements / bucket_size;           // WRONG, zero  
int buckets_req = div_to_inf(elements, bucket_size); // OK, one
```

- hardware division *always* rounds towards zero
 - only mode where $r = x - y * q$ doesn't overflow
- software division in other langs. sometimes supports other modes

Why does this need to be in the standard?

- users need it all the time, and try to implement it
- they fail miserably: almost all attempts on StackOverflow/blogs wrong



BUG: [\[StackOverflowCeil\]](#) answer (67 upvotes) fails at rounding to $+\infty$:

```
q = (x % y) ? x / y + 1 : x / y;
```

Mistake: incrementing negative quotients: $x=-1$, $y=2$, $q=1$

- pulling in third-party library for one division function is silly

Computing remainders is hard



BUG: The following function has unintended UB:

```
auto div_rem_ceil(_BitInt(4) x, _BitInt(4) y) {  
    _BitInt(4) q = div_ceil(x, y);    // round to +∞  
    _BitInt(4) r = x - y * q;        // overflow  
    return div_result<_BitInt(4)> { q, r };  
}
```

`div_rem_ceil(7, 2)` overflows: `q == 4`, `y * q == 8`

Conclusion: provide functions for computing quotient and remainder

Which rounding modes to support

<code>div_to_zero</code>  (trunc)	<code>div_ties_to_zero</code>
<code>div_away_zero</code>	<code>div_ties_away_zero</code>  (round)
<code>div_to_inf</code>  (ceil)	<code>div_ties_to_inf</code>
<code>div_to_neg_inf</code>  (floor)	<code>div_ties_to_neg_inf</code>
<code>div_to_even</code> 	<code>div_ties_to_even</code>  
<code>div_to_odd</code> 	<code>div_ties_to_odd</code> 

- `marked` functions are must-have, others fill the gaps
-  — unbiased |  — ISO/IEC 60559
- each `div_mode` function has `div_rem_mode` counterpart (24 total)

Library interface

```
template<class T> struct div_result
{ T quotient, remainder;
  friend constexpr auto operator<=>(< /* ... */> = default; };
template<class T> constexpr div_result<T> div_rem_mode(T x, T y);
template<class T> constexpr T          div_to_mode(T x, T y);
template<class T> constexpr T          mod(T x, T y);
```

- no `noexcept` because narrow contract (*Preconditions:*)
- no `rounding_mode` constant template- or run-time parameter
- `mod` is `div_rem_to_neg_inf(x, y).remainder`
 - as in $-2 \bmod 5 = 3$, and as in `mod` in Haskell, Ada, CSS, etc.

Other design considerations

- no SIMD overloads, but could be added in the future
 - branchless implementation possible
- no functions that compute only remainder (except mod)
 - these are not usually needed
 - undesirable remainder sign for most rounding modes
- all functions are in `<numeric>`
- feature-test macro: `__cpp_lib_integer_division`

Implementation experience

- Appendix A has simplified implementation for educational purposes
- eisenwave/integer-division [\[GitHub\]](#) repo has full implementation
- all functions can be made branchless and are suitable for SIMD port



EXAMPLE: Implementation is always in terms of / and %.

```
div_result<int> div_rem_to_neg_inf(int x, int y) {  
    bool quotient_negative = (x ^ y) < 0;  
    int adjust = int((x % y != 0) & quotient_negative);  
    return { x / y - adjust, x % y + adjust * y };  
}
```

Questions

- Agreement with overall design?
- SIMD overloads in another proposal?
- Forward to LEWG?

References

[P3724R1] *Jan Schultke*. Integer division (2025-09-29)

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3724r1.html>

[GitHub] *Jan Schultke*. integer-division GitHub repository

<https://github.com/Eisenwave/integer-division>

[StackOverflowCeil] Fast ceiling of an integer division in C / C++

<https://stackoverflow.com/q/2745074/5740428>